- **GitHub project URL:** https://github.com/Beastdzns/Kitchen-Display
- **Live app:** Not deployed.

Technical Design Document

Company A Direct Ordering and Kitchen Operations Platform

1. Purpose and scope

This project is a direct-ordering web application for the Hyderabad restaurant scenario in the project brief. It includes a customer ordering flow, outlet selection, kitchen display system (KDS), menu administration, basic analytics, catering requests, and demo AI-style preparation estimates and reorder suggestions.

The brief's annual recovery figures are scenario estimates, not measured results: shifting 12% of aggregator order value to direct ordering is modeled as ₹95.04 lakh annual margin recovery; online catering is modeled as ₹19.44 lakh; together the modeled impact is ₹1.1448 crore. These figures depend on the brief's stated costs, order counts, and margins and are not a guarantee of savings.

2. Architecture and rationale

The application uses Next.js App Router with React and TypeScript, Tailwind CSS, a custom Node.js HTTP server, Socket.IO, Drizzle ORM, and SQLite (`better-sqlite3`).

- **One local server:** `server.ts` serves the Next.js app and Socket.IO on the same port, simplifying the demo and avoiding separate application and real-time servers.
- **HTTP APIs and pages:** Next.js routes handle menu, order, catering, KDS, and administration requests.
- **Live updates:** Socket.IO rooms are used for outlet, order, and admin updates. KDS status changes can update customer tracking and administrative screens without a manual refresh.
- **Local persistence:** SQLite keeps the demo self-contained. Drizzle defines the application schema, and the database file is stored in the project directory.
- **Business logic modules:** Pricing, geographic outlet selection, preparation estimates, and reorder suggestions are kept in focused library modules and covered by tests.

[ASSUMPTION-01] The brief does not mandate a technology stack or hosting provider. The stack above is the project's implementation choice for a locally runnable demonstration; it is not a claim that it is the only suitable production architecture.

3. Data model

The schema in `src/lib/db/schema.ts` contains these entities:

Entity	Purpose and main relationships
`outlets`	Outlet or cloud-kitchen details, service radius, minimum preparation time, and capacity inputs.

Entity	Purpose and main relationships
`categories`	Menu category names and ordering.
`menu_items`	Dish details, price, portion, dietary flags, allergens, image path, and base preparation time.
`outlet_menu_overrides`	Outlet-specific availability and optional price overrides for menu items.
`users`	Demo/customer and staff role records, loyalty points, and consent state.
`orders`	Customer and outlet references, order status, totals, payment method, and promised time.
`order_items`	Dish, quantity, portion, and price snapshots associated with an order.
`party_orders`	Catering request, guest count, event date, estimated budget, assignment, and status.
`audit_notifications`	Simulated SMS/WhatsApp notification records associated with orders.
`loyalty_ledger`	Points changes associated with a user and, where applicable, an order.

The relationships support the main flows: users place orders and catering requests; orders belong to outlets and contain order items; menu availability can be overridden per outlet; and order events can produce notification and loyalty records.

4. Main flows

1. **Browse and route:** A customer chooses a locality or uses browser geolocation. The app selects a nearby outlet using the configured outlet coordinates and delivery radii.
2. **Build an order:** The menu is loaded for the selected outlet. Customers can filter dishes, view portion/allergen/dietary information, and add items to the cart. Reorder suggestions require an explicit customer action.
3. **Checkout:** The app calculates the order total, discounts, inclusive GST, and preparation estimate. The demo uses simulated payment choices; it does not process a real payment.
4. **Track and prepare:** An order appears on the KDS. Staff can advance its status, while Socket.IO broadcasts updates to the order-tracking view and other interested screens.
5. **Manage menu:** An administrator can change outlet availability (“86” an item). Availability changes are broadcast to menu clients.
6. **Handle catering:** A customer submits a party request subject to the configured minimum guest count and notice period. A catering manager reviews the request in the desk.

5. AI and automation

The current preparation-time feature is a deterministic heuristic, not a trained machine-learning model. For a non-empty order it:

1. Starts with the longest item's base preparation time, then adds a 15% item-time allowance for each item and two minutes for each quantity beyond one.
2. Calculates kitchen load as open tickets divided by staff count times the station-capacity factor. The load multiplier is `1 + min(load ratio, 2) × 0.6`.

3. Applies a 1.35 rush factor on Friday/Sunday from 19:30 to 22:30, a 1.15 factor during other days' same dinner window, or 1.0 otherwise.
4. Rounds the estimate, applies the outlet minimum, then adds any manager override.

The UI returns an explanation of the estimate. The separate reorder helper uses supplied order history and meal-time affinity to suggest items; it does not add them to the cart automatically.

[ASSUMPTION-02] The brief calls for AI-assisted preparation estimates and reorder suggestions but does not provide a trained model or training data. The rule-based heuristic is used as a transparent demo implementation. A production prediction model would require representative, consented operational data and validation.

[ASSUMPTION-03] Payment, SMS, and WhatsApp behavior is simulated because the project has no production payment credentials or notification-provider configuration.

6. UI and design decisions

The customer and operator screens use a dark plum/charcoal base with amber accents and high-contrast text. This connects the interface to the supplied royal Hyderabad/Diwali visual direction while keeping interactive controls distinct.

- **Home and outlet selection (`/`):** Locality chips and GPS selection make the service area the first decision. A supplied feast image is used as the hero backdrop, with the chicken biryani image retained as a dish visual.
- **Menu (`/menu`):** Search, category and vegetarian filters, dish photography, portion and dietary details, availability, and clear add controls support quick scanning.
- **Cart (`/cart`):** Order lines, discounts, loyalty redemption, GST disclosure, and consent are grouped into a single checkout flow.
- **Order tracking (`/order/[id]`):** Status and time information is presented as a progress view, with live updates from the server.
- **KDS (`/kds`):** Large status controls and urgency styling prioritize quick recognition and touch interaction in a kitchen.
- **Menu administration and analytics (`/admin/menu`, `/admin/dashboard`):** Availability/pricing controls and operational summaries are separated from customer ordering.
- **Catering (`/catering`, `/catering/desk`):** Customer request entry and manager review use distinct screens for submission and follow-up.
- **Branding and food photography:** The supplied diya emblem, hero feast, chicken biryani, Double Ka Meetha, and haleem images are served as optimized WebP files from `public/assets/`. Original PNGs remain in `assets/`.

7. Assumptions and limitations

- **[ASSUMPTION-04]** Outlet coordinates, contact details, and regulatory identifiers in the demo data are illustrative configuration and must be verified by the business before production use.
- **[ASSUMPTION-05]** Demo names, contact values, and delivery addresses are placeholders, not reviewer credentials or real customer records. The contact numbers use non-routable zero-prefixed placeholders.

- **[ASSUMPTION-06]** The demo uses a local SQLite database. A production multi-instance deployment would need persistent shared storage or a separate managed database and a compatible real-time deployment topology.
- **[ASSUMPTION-07]** The application has not been deployed for this submission. Production payment, delivery dispatch, and messaging integrations are outside the verified demo scope.

8. What was cut and what comes next

Cut from this submission: a trained/validated ML model, live payment processing, live SMS/WhatsApp delivery, external rider dispatch and tracking, and deployment to a production hosting service.

Next steps: validate outlet and regulatory data with the business; connect approved payment and notification providers; define a persistent production database and real-time hosting approach; collect operational data under an approved privacy policy; and evaluate a trained preparation-time model against the current heuristic.